Block Convey

PRISMtrace

by Block Convey

Getting Started Guide

Connect your AI agent, configure guardrails, and see every execution step — in under 15 minutes.

GETTING STARTED GUIDE | 2026

Block Convey

Connect your AI agent to PRISMtrace and get full visibility into every agent decision, guardrail block, and quality score — from day one.

WHAT PRISMTRACE GIVES YOU

OBSERVE

Every agent interaction visible — quality scores, latency, cost, guardrail flags, token counts.

→ Enables per-agent SLA enforcement; feeds data warehouse via export destinations.

PROTECT

PII/PHI detection and prompt injection blocking intercept threats before the LLM sees them.

→ Intercepts inputs before forwarding to the model — zero changes to LLM call logic.

TRACE

Trajectory shows every step your agent took — data retrieval, guardrail check, LLM call — with individual latencies.

→ Instrument via session_id + step + step_order metadata fields; no graph restructuring required.

GOVERN

Prompt versioning creates a full audit trail — who changed instructions, when, and the quality impact.

→ Tracks prompt_id, author, timestamp, and quality delta per version.

ALERT

Real-time alerts fire the moment quality drops or guardrail block rates spike.

→ Email delivery via Resend, recipients configurable per rule; Slack and PagerDuty webhooks on the roadmap.

REPORT

Auto-generated compliance reports mapped to NIST AI RMF, SR 11-7, NAIC, NYDFS, HIPAA AI, CFPB, EU AI Act.

→ Export as PDF or deliver to auditor portal — no manual data assembly.

1. GET YOUR API KEY (~2 MIN)

FROM THE DASHBOARD

- **API Keys** in the sidebar → Generate API Key → copy immediately. Shown once only.
- **Settings** → **Project tab** → copy your Project ID (UUID format).
- **Store both in a .env file.** Never put them in source code.

QUICK VALIDATION (60 SECONDS)

Before writing SDK code, confirm credentials work:

```
export PRISMTRACE_API_KEY="pt-sk-..."
export PRISMTRACE_PROJECT_ID="uuid..."

curl -X POST
https://prismtrace.blockconvey.com/api/traces \
  -H "Content-Type: application/json" \
  -d '{
    "api_key": "'$PRISMTRACE_API_KEY'",
    "project_id": "'$PRISMTRACE_PROJECT_ID'",
    "input_messages": [{"role":"user",
      "content":"Hello"}],
    "output_message": "Hi there!",
  "model":           "gpt-4o",
```

PYTHON — BASIC INTEGRATION

```
pip install blockconvey-monitor>=0.1.3
```

```
import os, requests
from dotenv import load_dotenv
load_dotenv()

def send_trace(user_input, output,
               model, latency_ms):
    requests.post(
        'https://prismtrace.blockconvey.com'
        '/api/traces',
        json={
          'project_id': os.getenv(
            'PRISMTRACE_PROJECT_ID'),
          'api_key': os.getenv(
            'PRISMTRACE_API_KEY'),
          'input_messages': [
            {'role':'user',
             'content': user_input}],
          'output_message': output,
          'model': model,
          'latency_ms': latency_ms,
          'metadata': {
```

```
  "latency_ms":      500
}'
```

Open Traces — your test trace appears within seconds.

```
            'agent_name': 'My Agent',
            'agent_id': 'my-agent-001'
          }
        }, timeout=10)
```

NON-BLOCKING (PRODUCTION)
Wrap send_trace() in a daemon thread to avoid blocking your agent:

```
import threading

def send_trace_async(user_input, output,
                     model, latency_ms):
    t = threading.Thread(
        target=send_trace,
        args=(user_input, output,
              model, latency_ms),
        daemon=True)
    t.start()

# Returns immediately
send_trace_async("Balance?",
    "$5,432.", "gpt-4o", 320)
```

✓ Always fire-and-forget. Never block the agent response path with a synchronous POST.

The LangGraph SDK integration takes three lines. Guardrails take 30 seconds to activate and protect your agents immediately.

2. CONNECT YOUR AGENT — LANGGRAPH SDK

THREE LINES — NO GRAPH CHANGES NEEDED

```
from blockconvey.integrations.langgraph
  import BlockConveyCallbackHandler

handler = BlockConveyCallbackHandler(
  api_key=os.getenv('PRISMTRACE_API_KEY'),
  project_id=os.getenv(
    'PRISMTRACE_PROJECT_ID'),
  agent_name='My Agent',
)

# Pass when invoking — graph unchanged
graph.invoke(state, config={
  'callbacks': [handler]})
```

✓ **That is the complete LangGraph integration.**
Every LLM call inside your graph now flows to PRISMtrace. Traces appear in the dashboard within seconds.

SUPPORTED INTEGRATIONS

- **LangChain** — callback handler, zero graph changes
- **LangGraph** — callback handler, zero graph changes
- **Anthropic SDK** — direct client wrapper
- **OpenAI SDK** — direct client wrapper
- **AWS Bedrock** — runtime wrapper
- **LiteLLM** — proxy wrapper
- **Node.js** — axios/fetch to REST endpoint
- **cURL** — REST endpoint directly

REQUIRED FIELDS IN EVERY TRACE

- **project_id** — your project UUID
- **api_key** — your pt_... API key
- **input_messages** — array of {role, content} objects
- **output_message** — string response from your agent

OPTIONAL BUT RECOMMENDED

- **model** — model name for cost tracking
- **latency_ms** — integer milliseconds
- token_count_input / token_count_output
- metadata.agent_name, metadata.agent_id

3. CONFIGURE GUARDRAILS AND ALERTS (~5 MIN)

BUILT-IN GUARDRAIL TYPES

- **PII/PHI Detection** SSNs, credit cards, emails, phone numbers, health data
- **Prompt Injection** Adversarial prompts attempting to override agent instructions
- **Content Moderation** Harmful or inappropriate language
- **Off-Topic Detection** Queries outside your agent's domain (Knowledge Base-linked; contact Block Convey support to enable for your project)
- **Custom Regex** Your own patterns — account numbers, internal codes
- **Language Restriction** Enforce response language constraints

✓ **Start here**
Enable PII Detection → Block and Prompt Injection → Block immediately. 30 seconds. Covers the most critical risks.

RECOMMENDED ALERT RULES

Metric	Condition	Why
Compliance Score	Below 80	Catches degradation early
Latency	Above 5000ms	Identifies slow responses
Guardrail Blocks	Above 10/hr	Detects attack patterns

QUALITY SCORES — EVERY TRACE

- **Customer Satisfaction (0–100)** How well the agent addressed the request
- **Response Quality (0–100)** Technical accuracy, completeness, clarity
- **Compliance Risk (0–100)** Lower = riskier. Flags missing disclaimers, PII leakage
- **Risk Level** Critical / High / Medium / Low composite

Block Convey

PROMPT LIBRARY

- Every system prompt edit creates a new version
- Side-by-side diff between any two versions
- One-click rollback to any prior version

Never put API keys or PII in prompts.
Use environment variables for secrets — not the Prompt Library.

Trajectory shows every step your agent took — making agent behavior explainable to engineers, compliance officers, and auditors.

4. ENABLE AGENT TRAJECTORY (OPTIONAL, ~10 MIN)

WHAT IS TRAJECTORY?

Trajectory is PRISMtrace's step-by-step execution view. When your agent queries a database, checks guardrails, and calls an LLM — trajectory shows each step in sequence with its latency and output.

Trajectory is NOT automatic.
Without the code below, the trajectory tab shows an empty state. This section shows exactly how to enable it.

THREE LEVELS

Level	What to Add	What You See
Basic (no code)	Nothing	Empty state
LLM-only (3 fields)	session_id + step + step_order	1-step: LLM C
Multi-step	One trace per step, shared session_id	Full sequence total latency

LEVEL 1 — LLM-ONLY (3 FIELDS, 2 MINUTES)

Add three fields to your existing trace metadata:

```
import uuid

# One UUID per user turn
turn_id = str(uuid.uuid4())

# Add to metadata in send_trace():
'metadata': {
  'agent_name': 'My Agent',
  'agent_id':   'my-agent-001',
  'step':        'llm_call',
  'step_order':  1,
```

LEVEL 2 — MULTI-STEP (FULL EXAMPLE)

Each step gets its own trace. All share the same session_id.

```
def handle_message(user_input):
  turn_id = str(uuid.uuid4())

  # Step 1 — database query
  t0 = time.time()
  data = your_db.query(user_input)
  ms0 = int((time.time()-t0)*1000)

  requests.post(TRACE_URL, json={
    ...,
    'model':       'databricks-query',
    'output_message': f'{len(data)} rows',
    'latency_ms':  ms0,
    'metadata': {
      'step':       'data_retrieval',
      'step_order': 1,
      'session_id': turn_id
    }})

  # Step 2 — LLM call
  t1 = time.time()
  resp = your_llm.call(data, user_input)
  ms1 = int((time.time()-t1)*1000)

  requests.post(TRACE_URL, json={
    ...,
    'model':          'claude-sonnet-4-...',
    'output_message': resp,
    'latency_ms':     ms1,
    'metadata': {
      'step':       'llm_call',
      'step_order': 2,
      'session_id': turn_id  # same!
    }})
  return resp
```

MODEL VALUES → TRAJECTORY LABELS

```
  'session_id':  turn_id,
}
```

Result: trajectory tab shows LLM Call with real latency.

- **databricks-query** Data Retrieval — Databricks
- **snowflake-query** Data Retrieval — Snowflake
- **guardrail** Guardrail Check
- **claude-sonnet-4-...** LLM Call — Claude Sonnet
- **gpt-4o** LLM Call — GPT-4o

✓ Compliance evidence

When a guardrail blocks, the trajectory ends there — LLM Call never appears. Timestamp, rule name, and matched text are recorded automatically for auditors.

Block Convey

Export trace data to your data warehouse, secure your API keys, and keep this page as a desk reference.

INTEGRATIONS | EXPORT DESTINATIONS

WHAT EXPORT DESTINATIONS DO

Export Destinations send your PRISMtrace trace data to your data warehouse so your data team can run SQL on it and build dashboards in their existing tools.

i Not for observing your database

These send data OUT of PRISMtrace. Observing database calls inside your agent uses trajectory instrumentation (Step 5 above).

- **Databricks** Writes to a Delta Lake table in your workspace
- **Snowflake** Writes to PRISMTRACE.PUBLIC.TRACES via MERGE
- **Amazon S3** Coming soon — archival and pipeline integration

AGENT FLEET

Every agent auto-detected from traces — no manual registration. Quality score, guardrail rate, latency, and last active shown per agent. Add description, owner, and environment via the pencil icon on any card.

EXAMPLE QUERY AFTER DATABRICKS EXPORT

```
SELECT agent_name,
  COUNT(*) AS traces,
  ROUND(AVG(quality_score),1)
    AS avg_quality,
  SUM(CASE WHEN guardrail_blocked
    THEN 1 ELSE 0 END) AS blocks
FROM prismtrace_traces
WHERE created_at >=
  DATE_SUB(NOW(), INTERVAL 7 DAY)
GROUP BY agent_name
ORDER BY blocks DESC;
```

SECURITY | API KEY RULES

✓ Store in .env or platform secret manager
✓ Add .env to .gitignore before first commit
✓ Rotate keys periodically
✓ Separate projects per environment
✗ Never hardcode keys in source code
✗ Never commit .env to Git

QUICK REFERENCE

Item	Value
Platform URL	https://prismtrace.blockconvey.com
API Endpoint	POST https://prismtrace.blockconvey.com/api/traces
SDK Install	pip install blockconvey-monitor>=0.1.3
API Key Location	API Keys page — shown once, copy immediately
Project ID Location	Settings → Project tab
Required Fields	project_id, api_key, input_messages, output_message
Trajectory Fields	metadata.session_id + metadata.step + metadata.step_order
Trajectory Model Names	databricks-query, snowflake-query, guardrail, or any LLM model name
Support	arunprasad@blockconvey.com — 24hr response on business days